

ATSAgent — agentic hiring plan

From requisition to released offer letter: a multi-agent system that does the work, with people deciding at the points that matter (human-in-the-loop, HITL).

Status: **proposal** (2026-10-05). Decisions already taken are marked ; open questions are in §12; a one-page inventory of agents, tools, skills, harness, loops and observability is in Appendix A.

1. Goal and principles

ATSIQ today is a governed workflow app with AI *features*: a person clicks, one model call runs, a person reads the result. ATSAgent turns that around — **agents move each requisition through the lifecycle on their own and stop for a person only when a decision needs one.**

Principles (non-negotiable):

1. **Agents act through the same doors as people.** Every agent action is a call to an existing server function (or a thin "tool" wrapper over its core logic) with the same `assertRole`, org scoping, transition tables and audit. No agent-only back doors, no direct SQL from the model.
2. **Governance stays human.** The approval chains already encoded in `REQ_TRANSITIONS` (`src/lib/requisitions.functions.ts:30`) and `OFFER_TRANSITIONS` (`src/lib/offers.functions.ts:31`), JD approval, and the pre-onboarding release gate are **always** human decisions. Agents prepare, summarise and chase; people approve.
3. **No silent rejection of candidates.** Agents may rank, shortlist and recommend; moving a candidate to `rejected` is a human decision (or an explicitly whitelisted, audited rule such as "missing work authorisation").
4. **Autonomy is a per-organisation dial, per agent**  — `suggest` → `act_and_notify` → `autonomous` — and the dial never overrides principle 2.
5. **Candidate contact is drafted by agents and sent after approval** , or auto-sent only for low-risk templates the organisation whitelists.
6. **Built on our own gateway**  — multi-vendor (OpenAI / Anthropic / Google), org-owned keys, every request in the `ai_usage_events` ledger, no vendor names on the wire. We extend `aiJson` with tool calling rather than adopting a third-party agent framework.
7. **Everything is replayable.** Every agent run, step, tool call, model output and human decision is persisted, so any outcome can be explained to a CHRO, a candidate or an auditor.

2. What exists today (the foundation we build on)

Lifecycle stage	Today	Code
Requisition	Manual create; <code>draft</code> → <code>pending_dh</code> → <code>pending_hr</code> → <code>pending_cbo</code> → approved with server-built approval trail; <code>on_hold</code> / <code>closed</code> / <code>rejected</code>	<code>requisitions.functions.ts</code> (<code>createRequisition</code> , <code>advanceRequisition</code>), <code>req_status</code> enum

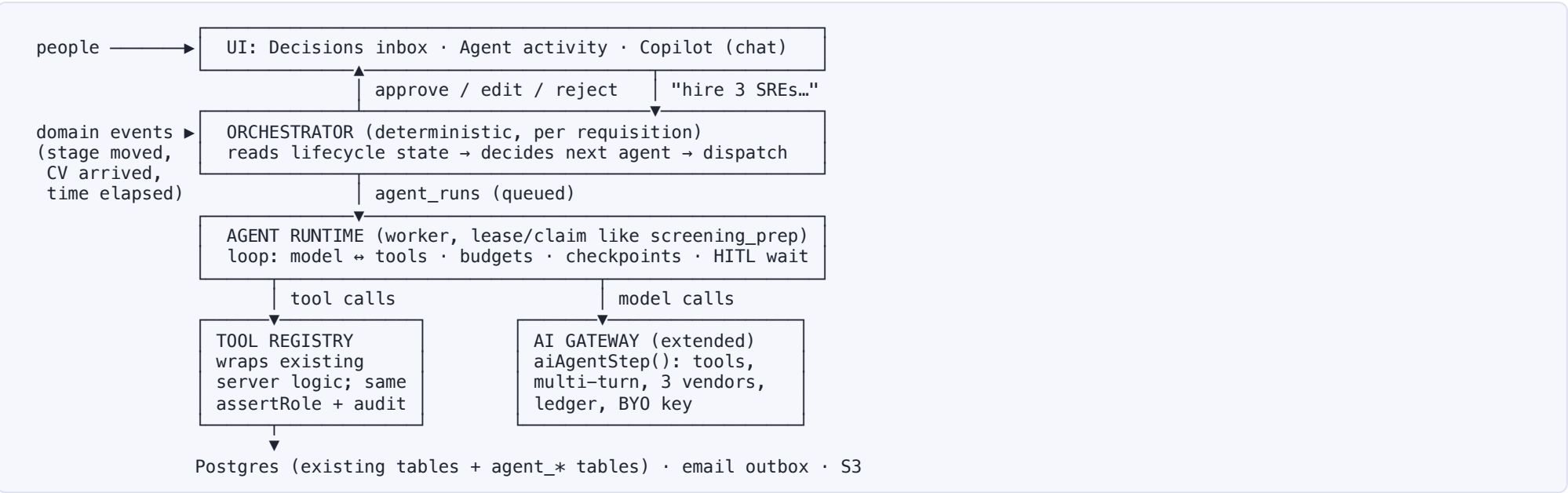
Lifecycle stage	Today	Code
JD	AI generate / import / parse; draft → pending_dh → approved / changes_requested; versioned; dedupe	saveJobDescription, approveJobDescription, jd-dedupe.ts; AI: jd_generate, jd_import, jd_parse
Scoring weights	AI-suggested weights per requisition	saveRequisitionWeights; AI: weight_suggest
Compensation	Live market research with citations, org corrections	market.server.ts, comp-knowledge.*; AI: market_benchmark, salary_research
Publishing	Careers/apply link, IJP, LinkedIn feed post, Indeed/Naukri/LinkedIn job boards (contract-gated)	setRequisitionIjp, src/server/boards/, AI: linkedin_post, jobcard_qa
Sourcing / intake	Apply page, careers-address inbound webhook, bulk CV parse, Capture extension, talent pool, referrals, talent requests	local-inbox.server.ts, capture.server.ts, intake.server.ts; AI: resume_parse
Matching	Explainable JD↔CV score, background autoscore + auto-shortlist, social-claim verification	autoscore.server.ts, matching.functions.ts; AI: candidate_score, claim_verify, *_signal
Screening	Background screening-kit prep on shortlist, triage queue, audio transcription, grading	screening-prep.server.ts, screening.*; AI: screening_kit, screening_grade, audio_transcribe
Assessment	Generated assessments + scoring via tokenised link	assess.\$token.tsx; AI: assessment_generate, assessment_score
Interviews	Scheduling with Meet / Teams / Zoom, interviewer portal, scorecards	interviews.functions.ts (scheduleInterview, submitScorecard)
Offer	draft → pending_hr → pending_cbo → approved → released → accepted/declined/revoked; AI letter from template; PDF	offers.functions.ts, offer-letter-pdf.ts; AI: offer_letter
Pre-onboarding (release gate)	Document collection, AI extraction, HR validation; release blocked until ready	onboarding.* (readinessFor); AI: doc_extract
Candidate comms	Email outbox (ack, stage update, interview invite, offer) with per-org toggles, retries	email-outbox.server.ts, process-email-outbox cron
"What needs me"	Live-derived action inbox (approval / interview / offer / invite / stale) + "Needs you today" strip	notifications.functions.ts
Copilot	Read-only Q&A over a JSON data snapshot; one aiJson call per message; no actions	copilot.functions.ts:179
Stage machine	Application stages + allowed transitions + reasons; single choke point journals every move	lifecycle.ts, stage-events.server.ts (recordStageTransition)
Background work	Lease/claim job tables + cron routes (screening_prep_jobs, email_outbox), bearer CRON_SECRET	src/routes/api/public/*, src/server/cron-auth.ts
Governance	audit_log + writeAudit, roles matrix, tenant predicates, untrusted() + INJECTION_RULES, zod-validated model output	auth.middleware.ts, src/server/audit.ts, docs/roles-and-rights.md

Gap summary. The pieces of an agent system are mostly here — state machines, approval rules, workers, an outbox, a ledger, injection defences. What is missing:

- **Tool calling / multi-step loops** in the gateway (it is single-shot JSON, ai-gateway.server.ts:342).

- **A runtime** that persists runs and steps, resumes after a human decision, and enforces budgets.
- **An orchestrator** that notices "this requisition is now X" and dispatches the right agent.
- **A decisions inbox** where agents put work in front of people and wait.
- **Per-org agent policy** (the autonomy dial, whitelists, budgets).

3. Target architecture



3.1 Orchestrator — deterministic, not an LLM

The orchestrator is ordinary code: a per-requisition **pipeline state** derived from the tables that already exist (requisition status, JD status, posting state, applicant counts per stage, interviews, offers). On every domain event it evaluates rules such as:

When	Dispatch
requisition created (or a hiring request arrives in chat)	Requisition agent
requisition <code>approved</code> , no approved JD	JD agent
requisition + JD approved, not published	Publishing agent

When	Dispatch
new application / CV arrives	Intake & matching agent
candidate <code>shortlisted</code>	Screening agent
screening graded "proceed"	Interview coordinator agent
all scorecards in for a round	Evaluation agent (debrief + recommendation)
recommendation <code>select</code> confirmed by hiring manager	Offer agent
offer <code>approved</code>	Pre-onboarding agent → release prep
anything stalled past SLA (<code>stalledDays</code>)	Follow-up agent (chase the human or candidate)

Why deterministic: routing must be explainable, testable and cheap; the model is used **inside** agents for judgement and drafting, never to decide who gets to approve what. An LLM "planner" is used only by the Copilot to turn a chat request into orchestrator commands (§4.10).

Events come from one place: a small `agent_events` outbox written by the existing choke points (`recordStageTransition` , `advanceRequisition` , `advanceOffer` , intake, scorecard submit) inside the same transaction, drained by the orchestrator worker. This reuses the outbox pattern of `email_outbox` .

3.2 Agent runtime

A worker (cron route `/api/public/agent-tick` , lease/claim exactly like `screening_prep_jobs`) that:

1. claims a queued `agent_run` , loads its checkpoint;
2. loops `model` → `tool calls` → `results` via the extended gateway until the agent returns a final answer, **requests a human decision**, or hits a budget (max steps, max tokens, wall-clock);
3. persists every step (`agent_steps`) before executing the next one, so a crash or deploy resumes cleanly;
4. on a HITL request, writes an `agent_task` , sets the run to `awaiting_human` , releases the lease; the human's answer re-queues the run with the decision appended to its context.

Long waits (approvals that take days) cost nothing: the run is just a row.

3.3 Gateway extension (`aiAgentStep`)

Add one function next to `aiJson` in `src/lib/ai-gateway.server.ts` :

```
aiAgentStep({
  orgId, feature,           // ledger slug, e.g. "agent_requisition"
  system, messages,        // multi-turn transcript (user/assistant/tool)
  tools,                   // [{ name, description, inputSchema (zod → JSON Schema) }]
}): Promise<{ ok: true; message; toolCalls: {id,name,args}[]; usage } | AiError>
```

- Implemented for all three providers' native tool-calling formats, behind the existing `resolveAiConfig` (org key only, no platform fallback).
- Every call logs to `ai_usage_events` (as today) with the agent feature slug plus `run_id` / `step` in the ledger detail.
- Tool arguments are zod-validated **before** execution; invalid args go back to the model as a tool error (bounded retries), mirroring the `aiJson` schema-retry.
- Untrusted text returned by tools (CVs, emails, documents) is wrapped with `untrusted(...)`; `INJECTION_RULES` stays in every agent system prompt; a tool result can never add tools or change the policy.
- No vendor/model names leave the server (existing invariant).

3.4 Tool registry

`src/server/agents/tools/*.ts` — each tool is `{ name, description, input: zod, risk: "read" | "write" | "external", run(ctx, input) }`.

- Tools call the **core logic** of existing server functions (refactor handlers into `*.server.ts` cores where needed so both the RPC and the tool call share one implementation, the same way `recordStageTransition` is shared today).
- `ctx` carries `orgId`, the **acting principal** (§6.1), `runId`, and the policy. Tenant predicates are applied by the core logic exactly as today.
- `risk` drives the HITL policy: `read` always runs; `write` and `external` (anything that leaves the org — email, board post, meeting invite) are checked against the org's dial for that agent (§5).

Initial tool surface (each maps to existing code):

Domain	Tools
Requisition	<code>get_requisition</code> , <code>draft_requisition</code> , <code>submit_requisition_for_approval</code> , <code>suggest_weights</code> , <code>research_compensation</code> , <code>find_similar_roles</code>
JD	<code>generate_jd</code> , <code>revise_jd</code> , <code>submit_jd</code> , <code>dedupe_jd</code> , <code>get_template</code>
Publishing	<code>draft_linkedin_post</code> , <code>publish_to_board</code> , <code>enable_ijp</code> , <code>job_card_qa</code>
Pipeline	<code>list_applications</code> , <code>get_candidate</code> , <code>score_candidate</code> , <code>verify_claims</code> , <code>move_stage</code> (non-reject), <code>propose_rejection</code> , <code>search_talent_pool</code>
Screening	<code>build_screening_kit</code> , <code>grade_screening</code> , <code>send_assessment</code>
Interviews	<code>find_slots</code> , <code>schedule_interview</code> , <code>remind_interviewer</code> , <code>summarise_scorecards</code>
Offer	<code>draft_offer</code> , <code>generate_offer_letter</code> , <code>submit_offer</code> , <code>onboarding_readiness</code> , <code>request_documents</code> , <code>propose_release</code>
Comms	<code>draft_candidate_email</code> , <code>send_candidate_email</code> (via outbox), <code>notify_member</code>
HITL	<code>request_approval</code> , <code>ask_human</code> (clarifying question), <code>handoff</code> (give up with reason)

4. The agents

Each agent = system prompt + tool subset + output contract + HITL points. All share the runtime. "Gate" = a decision that is **always** human regardless of the autonomy dial.

4.1 Requisition agent

- **Trigger:** a hiring request (chat, form, or HRMS replacement signal).
- **Does:** fills the requisition from intent + org history (similar past requisitions, departments, masters), suggests openings/experience/skills, scoring weights, CTC band from live market research with citations, and flags conflicts (budget above band, duplicate open req).
- **Gate:** submitting is fine; **DH / HR head / CBO approvals** stay human. The agent prepares an approval brief for each approver (why this role, band evidence, risks) and chases approvers past SLA.

4.2 JD agent

- **Trigger:** requisition approved without an approved JD; or "changes requested".
- **Does:** drafts from org templates + requisition, dedupes against the JD library, revises on DH feedback, keeps `syncRequisitionFromJd` consistent.
- **Gate:** **JD approval** by department head.

4.3 Publishing agent

- **Trigger:** requisition + JD approved.
- **Does:** job-card QA, LinkedIn/board copy, IJP posting per org rules, publishes to connected boards (capability-gated as today).
- **HITL:** `external` — first post per requisition needs approval unless whitelisted.

4.4 Intake & matching agent

- **Trigger:** new application from any channel.
- **Does:** parse, dedupe, verify claims, score with evidence, place in `ai_screened` / `shortlisted` per the requisition's thresholds, propose rejections with reasons, and pull matching people from the talent pool and referrals when the funnel is thin.
- **Gate:** **rejection** is a proposal (`propose_rejection`) the recruiter confirms in bulk; shortlisting follows the dial.

4.5 Screening agent

- **Trigger:** `shortlisted`.
- **Does:** builds the screening kit (already backgrounded), sends the assessment, grades answers / transcripts, writes a proceed/hold summary.
- **HITL:** sending the assessment = candidate contact → approval or whitelisted template.

4.6 Interview coordinator agent

- **Trigger:** screening "proceed"; each round completed.
- **Does:** proposes panel from the requisition and past interviewers, finds slots, creates Meet/Teams/Zoom invites, sends candidate invites, reminds interviewers, chases missing scorecards.

- **HITL:** candidate invite → approval or whitelisted; panel choice → dial.

4.7 Evaluation agent

- **Trigger:** all scorecards in for a round.
- **Does:** debrief summary across scorecards, screening and match evidence, contradictions between interviewers, a `select / hold / reject` recommendation with reasons, and a bias check (`scripts/bias-report.ts` logic) across the requisition's funnel.
- **Gate:** the **hiring decision** is the hiring manager's.

4.8 Offer agent

- **Trigger:** `select` confirmed.
- **Does:** proposes CTC within band using market research and internal parity, drafts the offer, generates the letter from the org template, submits for approval with a justification brief.
- **Gate:** **HR head and CBO approvals**; the agent never edits an offer after submission except through the `draft` path.

4.9 Pre-onboarding & release agent

- **Trigger:** offer `approved`.
- **Does:** requests documents, extracts and cross-checks them (identity, education, previous CTC vs claims), lists what HR must validate, and when `readinessFor` is green prepares the release.
- **Gate:** **document validation** and **offer release** stay with the HR head (existing release gate).

4.10 Copilot (orchestrator front door)

The existing copilot (`copilot.functions.ts`) becomes conversational control: "Open 3 backend roles for the Bengaluru platform team, similar to last quarter's" → the copilot plans with **read tools +** `start_agent / explain_run / pause_run`, shows the plan, and on confirmation hands commands to the orchestrator. It keeps the guided-onboarding behaviour from `src/lib/user-manual.ts` (per AGENTS.md product guidance).

4.11 Follow-up agent

Watches SLAs (`stalledDays`, approval ageing, missing scorecards, unanswered candidates) and nudges the right person or drafts the candidate follow-up. Mostly deterministic; uses the model only to draft messages.

5. Human-in-the-loop model

5.1 Three kinds of human touchpoint

Kind	Example	Blocks the run?
Gate	requisition/JD/offer approval, release, rejection, hiring decision	yes — always human
Action approval	send this email, publish this post, book this panel	per autonomy dial
Clarification	"Two open reqs match — which one?"	yes, until answered or timed out

5.2 Autonomy dial (per org, per agent) ✓

Level	<code>write</code> tools	<code>external</code> tools (leave the org)
<code>suggest</code>	proposal → approve	proposal → approve
<code>act_and_notify</code>	run, notify owner, 1-click undo where reversible	proposal → approve, unless template whitelisted
<code>autonomous</code>	run	run for whitelisted templates; else approve

Gates (§5.1) ignore the dial. Agents are **opt-in**: every agent is off until the organisation switches it on, and starts at `suggest`, so value is visible before trust is extended.

5.3 Decisions inbox

One place where agents' requests land, extending the live action inbox in `notifications.functions.ts` with `kind: "agent"`:

- card = what the agent wants to do, why (evidence, sources), what happens on approve / edit / reject, and the run's trail;
- **edit-then-approve** for drafts (email, JD, offer letter);
- bulk approve for repetitive items (e.g. 40 proposed rejections, each with reasons);
- routed by role (same matrix as `docs/roles-and-rights.md`), with SLA escalation to the next approver;
- decisions are written to `audit_log` and fed back to the agent run.
- a gate tied to a requisition or JD performs the real approval step when decided in the inbox, as the deciding person (their role is checked by the same lifecycle core as the requisition page); a decision made on the regular pages closes the matching inbox request.
- a rejection batch lists every candidate with a reason tied to the requisition's stated requirements; approving rejects them as the deciding person (anyone who moved meanwhile is skipped), declining rejects nobody.
- a hiring decision goes to the hiring manager with the Evaluation agent's recommendation and rationale; approving a reject or hold moves the candidate as the decider, approving a select records it and hands it to the offer stage, declining changes nothing.

5.4 Kill switches

Per-org "pause all agents", per-requisition pause, per-run cancel. Platform super admin can disable an agent type platform-wide.

6. Security and compliance (maps to AGENTS.md invariants)

6.1 Agent identity

Each run executes **on behalf of a human principal** (the person who started it, or the requisition owner for event-triggered runs) and is labelled as an agent: `actor = "agent:<type>:<runId>"` with `on_behalf_of = userId`.

- `assertRole` is checked against the principal — an agent can never do what its principal could not.
- Gates additionally require the approving human's own session (the agent can't approve even if the principal has the role).
- `audit_log` gets `actor = agent`, `detail.on_behalf_of`, `detail.run_id`.

6.2 Invariants

- **Tenant scoping:** tools only reach data through core logic that already carries `eq(table.orgId, ctx.orgId)`; the runtime refuses a run whose context org differs from the row's org.
- **Untrusted text:** CV, JD, profile, mail and document content enter prompts only via `untrusted(...)`; tool outputs that carry such text are wrapped too; `INJECTION_RULES` is in every agent system prompt; a detected injection marks the candidate (`suspected_prompt_injection`) and forces `suggest` for that run.
- **Gate guard:** the tool registry refuses to register any tool whose name approves, releases, rejects, hires, revokes, declines or accepts — agents only get `propose_*`, `request_*` and `submit_*` variants, so a gate action can never be offered to a model.
- **Server-built trails:** approval trails stay server-built; agents can't write trail entries.
- **safeFetch only** for any URL a tool fetches.
- **Vendor neutrality:** agent UIs show "the agent", never vendor/model.
- **Ledger:** every model call through the gateway with an `agent_*` feature slug; per-org monthly agent budget with hard stop.
- **Rate limits:** the new cron route sits behind `CRON_SECRET`; new user RPCs inherit the server-fn limiter.

6.3 Agent governance (traceability)

Every agent is defined by a versioned **manifest** (identity, version, accountable owner, responsibility, permission scope, must-never list, gates it may request, risk tier, eval suite, instructions, tools and the AI skills those tools use).

- **Version per run:** each manifest version is stored once (`agent_definitions`, keyed by content hash); every run records the definition it executed under, and a change mid-run is recorded and audited.

- **Change control:** `scripts/agents.lock.json` pins each agent's version and hash; CI fails if a manifest changes without a version bump, or if a live agent has no real-agent eval.
- **Attribution:** every AI request inside a run — model turns and AI calls inside tools — carries the run id in the AI ledger.
- **Budgets:** an agent over its monthly token budget pauses (not fails), is audited once and is surfaced to HR leadership.
- **Audit:** run started / completed / failed / definition changed / budget paused / exported, plus every write or external action and every human decision.
- **Register and export:** Governance → Agent register shows each manifest, version history and activity; any run's complete trail (definition snapshot, steps, decisions, audit, AI requests without vendor names) exports as JSON.

6.4 Fairness and explainability

- No automated final rejection (principle 3); every proposed rejection carries reasons tied to the requisition's stated requirements.
- Scores keep their existing evidence/explanations; the evaluation agent runs the bias report per requisition and flags skew to HR.
- Candidate-facing messages come from approved templates or approved drafts.

7. Data model additions

Following the repo rule: `drizzle/schema.ts` + idempotent SQL in `drizzle/pg-migrations/` (next: `0024`) + regenerate `docs/er-diagram.md`.

Table	Purpose
<code>agent_policies</code>	per org × agent type: <code>autonomy</code> , whitelisted templates, budgets, enabled
<code>agent_events</code>	domain-event outbox written by choke points; drained by orchestrator (lease/claim)
<code>agent_runs</code>	one run: org, agent type, subject (requisition/application/offer id), principal, status (<code>queued</code> / <code>running</code> / <code>awaiting_human</code> / <code>done</code> / <code>failed</code> / <code>cancelled</code>), checkpoint, budget used, lease
<code>agent_steps</code>	ordered steps: model message, tool call (name, args), tool result (redacted), usage, duration, error
<code>agent_metrics_daily</code>	per org × agent × day rollup: runs, outcomes, steps, tokens, cost, p50/p95 latency, HITL wait, human-edit rate (feeds §9 dashboards cheaply)
<code>agent_tasks</code>	HITL items: kind (gate / approval / clarification), assignee role/user, payload, proposed action, decision, decided_by, decided_at, SLA

8. UI changes

- **Decisions inbox** (new page + dashboard strip), §5.3.
- **Agent activity** on requisition, candidate and offer pages: a timeline of what agents did and why, with links to steps and the decision that unblocked them.
- **Agent settings** (Integrations → Agents): the autonomy dial per agent, template whitelist, budget, pause.

- **Copilot** upgraded from Q&A to plan-and-confirm (\$4.10).
- Platform console: agent runs and spend across tenants (extends `/platform-ai-usage`).

9. Observability

Today the app has an AI spend ledger (`ai_usage_events`: feature, provider, model, tokens, attempt, latency per request), `audit_log` for privileged actions, the platform AI-usage console (`/platform-ai-usage`) and plain `console.error`. There is no structured logger, request id, tracing or alerting. Agents make this non-optional: a single hire becomes dozens of model calls, tool calls and human waits spread over weeks, and every one of them must be explainable.

9.1 What we capture

Signal	Contents	Where
Traces	one trace per agent run: run → steps → model call / tool call / HITL wait spans, linked to the triggering <code>agent_event</code> and to the requisition / application / offer	<code>agent_runs.trace_id</code> , <code>agent_steps</code> (span id, parent, timings)
Structured logs	JSON lines with <code>trace_id</code> , <code>run_id</code> , <code>org_id</code> , <code>agent</code> , <code>tool</code> , level, message; request id on every server-fn/route call	new <code>src/server/log.ts</code> ; stdout → Cloud Logging
Metrics	runs started/finished/failed, steps per run, tool error rate, tokens and cost, p50/p95 latency, HITL wait time, gate blocks, retries, injection detections	<code>agent_metrics_daily</code> rollup + log-based metrics
Quality	human-edit rate, approval vs rejection of proposals, override reasons, recommendation-vs-decision agreement	derived from <code>agent_tasks</code> decisions
AI spend	per call, as today, plus <code>run_id</code> / <code>step</code> so cost rolls up to a run, requisition, agent and org	<code>ai_usage_events</code> (extended)
Audit	who (agent on behalf of whom) did what, every gate decision	<code>audit_log</code> (unchanged invariant)

9.2 Where people see it

View	Audience	Shows
Agent activity timeline	recruiter, HR, hiring team	on each requisition / candidate / offer: what each agent did, why, evidence, and which human unblocked it
Run inspector	HR head, org owner	one run step by step: prompts (redacted), tool calls, results, decisions, cost, timings; replay button
Org agent dashboard	HR head, CHRO, org owner	throughput per stage, time saved, approval wait times, human-edit rate per agent, spend vs budget
Platform agent console	platform super admin	runs, failures, latency and spend across tenants (extends <code>/platform-ai-usage</code>); vendor detail stays here only

9.3 Alerts

Alert	Sent to
run stuck (lease expired twice, or running past wall-clock budget)	platform admin
tool or model error rate above threshold for an agent	platform admin
org at 80% / 100% of monthly agent budget	org owner, HR head
prompt-injection detected in candidate content	HR head
HITL task past SLA (after escalation)	next approver, HR head
gate-crossing attempt blocked	platform admin (security)

9.4 Rules

- **Redaction first:** traces and logs carry ids and short redacted summaries, never full CV, document or email text; full payloads stay in their source tables under existing access rules.
- **Tenant-scoped:** org views only ever query their own `org_id` ; the platform console aggregates without exposing tenant content.
- **Vendor-neutral:** model / vendor names appear only in the platform console and the org's own AI settings, as today.
- **Retention:** step payload detail kept for a fixed window (see §12), metrics and audit kept long-term.
- **Portable:** the logger and span model follow OpenTelemetry conventions so an org or the platform can export to its own backend later without code changes.

10. Phased delivery

Each phase ships end-to-end and keeps the app working without agents.

Phase 0 — Foundations

- `aiAgentStep` with tool calling for OpenAI, Anthropic, Google; ledger slugs
- `agent_*` tables (migration `0024`), runtime worker + `/api/public/agent-tick`
- tool registry skeleton + agent identity in `audit_log`
- `agent_policies` + Agents settings page (default `suggest`)
- Decisions inbox (gate / approval / clarification cards)
- observability foundation (§9): structured logger with trace ids, run/step traces, agent metrics, org agent-activity view
- eval harness (§11) with fixtures from `scripts/local-e2e/` and `test_data/`

Phase 1 — Requisition → JD → publish

- `agent_events` emitted from requisition/JD transitions; orchestrator v1
- Requisition, JD and Publishing agents; approval briefs + SLA chasing
- Copilot "open a role" → plan → confirm

Phase 2 — Intake → match → screen

- Intake & matching agent (wraps autoscore; proposed rejections in bulk)
- Screening agent (wraps screening-prep; assessment sending via approval)
- Follow-up agent for unanswered candidates

Phase 3 — Interviews → evaluation

- Interview coordinator (panel, slots, invites, reminders, scorecard chasing)
- Evaluation agent (debrief, recommendation, bias check)

Phase 4 — Offer → pre-onboarding → release

- Offer agent (band-aware proposal, letter, approval brief)
- Pre-onboarding & release agent (documents, cross-checks, release prep)

Phase 5 — Hardening and scale

- autonomy upgrades per org based on measured approval/edit rates
- platform console for agent runs; cost and latency tuning; replay tooling
- alerting (stuck runs, error spikes, budget, injection detections) and optional OpenTelemetry export to the org's own tracing backend

11. Evaluation and quality

- **Golden scenarios** per agent (fixture org, requisitions, CVs incl. prompt-injection samples) run in CI against a recorded or test key; assert tool sequences, gate compliance and output schemas.
- **Gate compliance tests**: for every gate, prove an agent run cannot cross it (role check, session check, transition table).
- **Human-edit rate** per agent and template (how often people change or reject what the agent proposed) is the main quality metric and the input for raising autonomy.
- **Replay**: any `agent_run` can be re-executed against a new prompt/model in a sandbox org and diffed.

12. Open questions

1. Which org pilots Phase 1, and what are its approver SLAs?
2. Default rejection rules that may be automated (e.g. missing work authorisation, notice period above `maxNoticePeriodDays`) — or none?

3. Should agents run 24x7 or only within the org's working hours for candidate-facing actions?
4. Budget policy: per-org monthly agent token cap and what happens at the cap (pause vs degrade to [suggest](#)).
5. Data retention for [agent_steps](#) (they contain redacted CV-derived text).

Appendix A — Inventory at a glance

Phases: P0 foundations · P1 requisition→JD→publish · P2 intake→match→screen · P3 interviews→evaluation · P4 offer→pre-onboarding→release · P5 hardening.

A.1 Agents

#	Agent	Starts when	What it does	Always needs a person for	Phase
1	Orchestrator (plain code)	any domain event	works out where each requisition is and starts the right agent	— (routes work only)	P0/P1
2	Copilot (chat front door)	a person types a request	turns it into a plan, shows it, hands it to the orchestrator once confirmed	confirming the plan	P1
3	Requisition agent	hiring request (chat, form, HRMS replacement)	fills the requisition, skills, weights, pay band with evidence; approval brief; chases approvers	DH / HR head / CBO approvals	P1
4	JD agent	requisition approved without JD, or changes requested	drafts from templates, dedupes, revises on feedback	DH JD approval	P1
5	Publishing agent	requisition + JD approved	job-card QA, LinkedIn / board copy, posts to boards and IJP	first external post (unless whitelisted)	P1
6	Intake & matching agent	new application	parse, dedupe, verify, score, shortlist; searches talent pool when funnel is thin	rejections (proposed, confirmed in bulk)	P2
7	Screening agent	candidate shortlisted	screening kit, assessment, grading, proceed / hold summary	sending assessment (unless whitelisted)	P2
8	Interview coordinator	screening "proceed" / round done	panel, slots, Meet / Teams / Zoom, candidate invite, scorecard chasing	candidate invite (unless whitelisted)	P3
9	Evaluation agent	all scorecards in for a round	debrief, disagreements, select / hold / reject recommendation, bias check	hiring decision (hiring manager)	P3
10	Offer agent	selection confirmed	pay within band, offer draft and letter, approval brief	HR head and CBO offer approvals	P4
11	Pre-onboarding & release agent	offer approved	requests and cross-checks documents, prepares release	document validation and release (HR head)	P4
12	Follow-up agent	anything past its deadline	nudges approvers / interviewers, drafts candidate follow-ups	candidate messages (unless whitelisted)	P2

A.2 Tools

Read tools always run; write and external tools follow the org's autonomy dial.

Area	Tools	Type	Built on
Requisition	<code>get_requisition</code> , <code>draft_requisition</code> , <code>submit_requisition_for_approval</code> , <code>suggest_weights</code> , <code>research_compensation</code> , <code>find_similar_roles</code>	read / write	<code>requisitions.functions.ts</code> , <code>market.server.ts</code>
JD	<code>generate_jd</code> , <code>revise_jd</code> , <code>submit_jd</code> , <code>dedupe_jd</code> , <code>get_template</code>	write	<code>saveJobDescription</code> , <code>jd-dedupe.ts</code>
Publishing	<code>draft_linkedin_post</code> , <code>publish_to_board</code> , <code>enable_ijp</code> , <code>job_card_qa</code>	external	<code>src/server/boards/</code> , <code>setRequisitionIjp</code>
Pipeline	<code>list_applications</code> , <code>get_candidate</code> , <code>score_candidate</code> , <code>verify_claims</code> , <code>move_stage</code> (no reject), <code>propose_rejection</code> , <code>search_talent_pool</code>	read / write	<code>autoscore.server.ts</code> , <code>lifecycle.ts</code> , <code>stage-events.server.ts</code>
Screening	<code>build_screening_kit</code> , <code>grade_screening</code> , <code>send_assessment</code>	write / external	<code>screening-prep.server.ts</code> , <code>screening.*</code>
Interviews	<code>find_slots</code> , <code>schedule_interview</code> , <code>remind_interviewer</code> , <code>summarise_scorecards</code>	write / external	<code>interviews.functions.ts</code>
Offer	<code>draft_offer</code> , <code>generate_offer_letter</code> , <code>submit_offer</code> , <code>onboarding_readiness</code> , <code>request_documents</code> , <code>propose_release</code>	write / external	<code>offers.functions.ts</code> , <code>onboarding.*</code>
Comms	<code>draft_candidate_email</code> , <code>send_candidate_email</code> (outbox), <code>notify_member</code>	external	<code>email-outbox.server.ts</code>
HITL	<code>request_approval</code> , <code>ask_human</code> , <code>handoff</code>	—	new: <code>agent_tasks</code>
Control	<code>start_agent</code> , <code>pause_run</code> , <code>explain_run</code> (Copilot only)	write	new: runtime

A.3 Skills (shared AI capabilities)

Skill	Used by	Status
CV parsing	Intake	exists (<code>resume_parse</code>)
JD writing / import / parsing	JD agent	exists (<code>jd_generate</code> , <code>jd_import</code> , <code>jd_parse</code>)
Scoring-weight suggestion	Requisition	exists (<code>weight_suggest</code>)
Explainable JD↔CV scoring	Intake & matching	exists (<code>candidate_score</code>)
Claim and social-signal verification	Intake & matching	exists (<code>claim_verify</code> , <code>linkedin_signal</code> , <code>writing_signal</code>)
Market pay research with citations	Requisition, Offer	exists (<code>market_benchmark</code> , <code>salary_research</code>)
Screening kit + grading	Screening	exists (<code>screening_kit</code> , <code>screening_grade</code>)
Audio transcription	Screening	exists (<code>audio_transcribe</code>)

Skill	Used by	Status
Assessment generation + scoring	Screening	exists (<code>assessment_generate</code> , <code>assessment_score</code>)
LinkedIn copy + job-card check	Publishing	exists (<code>linkedin_post</code> , <code>jobcard_qa</code>)
Offer letter drafting	Offer	exists (<code>offer_letter</code>)
Document extraction	Pre-onboarding	exists (<code>doc_extract</code>)
Approval brief (why, evidence, risks)	Requisition, Offer	new
Interview debrief + recommendation	Evaluation	new
Bias / fairness check	Evaluation	new (from <code>scripts/bias-report.ts</code>)
Candidate message drafting	Follow-up, Interviews, Screening	new
Plan from chat request	Copilot	new

A.4 Harness (P0)

Component	What it is	Notes
Gateway tool calling (<code>aiAgentStep</code>)	gateway extended with tools and multi-turn conversation	3 vendors, org's own key, usage ledger, no vendor names
Agent runtime worker	claims queued runs and drives them	lease / claim like screening-prep; <code>/api/public/agent-tick</code>
Tool registry	tools with checked inputs and read / write / external label	existing role checks and org scoping on every call
Policy engine	autonomy dial, template whitelist, budget	gates override every setting
Agent identity	"agent acting for this person"	logged in <code>audit_log</code> ; never exceeds that person's rights
Event outbox (<code>agent_events</code>)	domain events from existing choke points	drained by the orchestrator
Run + step store	<code>agent_runs</code> , <code>agent_steps</code>	resume after crash; replay
Decisions inbox	<code>agent_tasks</code> + UI	edit-then-approve, bulk approve, escalation
Guards	injection defence, I/O validation, step / token / time limits, kill switches	reuses <code>untrusted()</code> , <code>INJECTION_RULES</code>
Observability	logger, traces, metrics rollup, dashboards, alerts (§9)	redacted, tenant-scoped, OpenTelemetry-shaped
Eval harness	golden scenarios, gate-compliance tests, replay	runs in CI

A.5 Loops

Loop	Runs	Cycle	Stops when
Orchestrator loop	every tick (cron)	new events → requisition state → queue the right agent	no new events
Agent loop	per run	model → tool calls → results → model ...	final answer, needs a person, or budget hit
HITL loop	per decision	agent asks → run pauses (no cost) → person approves / edits / rejects → run resumes	decision made, or timeout and escalation
Schema-retry loop	per model call	bad output or tool input → error back to the model → retry	valid output, or retry limit
Deadline / follow-up loop	every tick	find stalled approvals, scorecards, candidate replies → nudge	nothing overdue
Outbox loop (exists)	every 5 min	send queued emails with retries	queue empty
Telemetry rollup loop	hourly / nightly	steps + usage + decisions → <code>agent_metrics_daily</code> → alerts	—
Learning loop	weekly / monthly	human-edit and rejection rates per agent → raise or lower autonomy	—

Source: docs/agentnic-plan.md · generated 2026-10-06